AI-BASED DETECTION OF FAKE OR MANIPULATED ACADEMIC CERTIFICATES

Phase-1 Project Progress Report, Evaluation Deliverable & Viva Voce Defense

Institutional Milestone: Final Year Engineering / Capstone Phase-1 (Weeks 1–16) • **Target Completion:** 70% • **Actual Progress:** 72%
Architecture: Decoupled Multi-Tiered Microservice (React 18 + Vite + FastAPI + PyTorch MobileNetV2 + Scikit-Learn)
Document Generated: November 12, 2026 • **Status:** Approved for Viva Voce Review

---

1. Literature Survey and Requirement Analysis

1.1 Problem Statement

Educational institutions, corporate employers, and government agencies face a critical credential fraud crisis fueled by modern digital editing and generative inpainting tools that manipulate academic certificates with near-undetectable visual fidelity. This module is built for admissions officers, background verification agencies, and HR recruiters who currently rely on slow manual inspections (taking 3 to 14 days) or unverified smartphone QR scans. It matters because counterfeit certificates undermine academic meritocracy, lead to unlawful employment and admissions, and cannot be caught by visual checks when pixel-level edits, font cloning, or QR-swapping attacks are employed.

1.2 Literature Survey

Paper (Author, Year, Venue)	Approach / Technique	Strength	Limitation
Anandamurugan et al. (2026) *IEEE Trans. Comput. Soc. Syst.*	**DIAR-AI:** Structural Similarity Index (SSIM ≥ 0.85) and Canny edge XOR difference (≤ 0.08) against templates.	Zero training requirement; fast execution (<25ms on CPU); highly effective for layout and border integrity.	Fails to detect localized font tampering inside valid text boxes if global geometry matches the template.
Khormi et al. (2026) *IEEE Access*	**Behavioral Anomaly Model:** Regularized Logistic Regression with 5-fold Stratified CV and SMOTE on issuance logs.	High precision (92.9%) and ROC-AUC (0.940); catches unauthorized off-hours insider issuance in <10ms.	Analyzes administrative server access logs only; completely blind to the visual and physical certificate pixels.
Verdoliva et al. (2024) *IEEE Trans. Inf. Forensics Secur.*	**MobileNetV2 CNN:** Depthwise separable convolutions with frozen early feature extractors for pixel tampering.	High tamper detection (>95% F1-score); compact model footprint (<3.5M params, ~9.5MB); runs under 60ms on CPU.	Prone to false alarms on real-world certificates subjected to benign multi-generational JPEG compression.
Huang et al. (2022) *ACM MM '22*	**LayoutLMv3:** Multimodal Transformer pre-training combining text tokens, 2D layout, and visual patch embeddings.	State-of-the-art semantic comprehension of complex tabular marks cards and hierarchical key-value relationships.	Heavy compute overhead (>2GB VRAM); latency exceeds 1500ms; requires discrete GPUs unfeasible for edge use.

1.3 Literature Gaps

1. The QR-Code Hijacking Blindspot: Existing commercial verifiers inspect only the payload inside the QR code. Attackers exploit this by copying a genuine student's valid QR code and pasting it onto a forged certificate with an altered name; standard QR scanners report 'Valid University URL' because they do not cross-examine the surrounding text.

2. Single-Model Brittleness: Pure deep learning CNNs produce unacceptably high false-positive rates on legitimately compressed or scanned paper documents, while pure OCR engines extract text without verifying if the underlying typography was spliced.

3. Addressed by Our Module: Our module bridges these gaps via **Automated Multimodal Cross-Verification**: cross-referencing decoded QR payloads against OCR-extracted text using fuzzy string token matching, while corroborating pixel Error Level Analysis (ELA) with template Structural Similarity (SSIM) and gradient-boosted decision trees.

1.4 Requirements Specification

ID	Requirement Specification	Type	Priority
R1	Ingest PDF, PNG, JPG, and TIFF certificates (≤10MB), validate magic byte signatures, and compute a SHA-256 digest to detect duplicates in <5ms.	Functional	High
R2	Deskew images within ±0.5°, apply CLAHE contrast enhancement, extract text tokens via Tesseract OCR, and parse candidate name, USN, course, and marks using regex.	Functional	High
R3	Execute Error Level Analysis (ELA at 90% JPEG re-save) and SIFT/ORB keypoint clustering to detect cloned stamps, signatures, or pasted text blocks.	Functional	High
R4	Decode 2D QR codes, validate URLs against an accredited domain whitelist, and verify that QR payload matches OCR-extracted credentials (≥85% match).	Functional	High
R5	Extract a standardized 22-dimensional feature vector, predict manipulation risk via regularized gradient-boosted decision trees, and generate an Authenticity Score (0-100%).	Functional	High
R6	Complete verification pipeline execution in <1500ms on standard multi-core commodity CPUs without requiring dedicated GPU acceleration.	Non-Functional	High
R7	Isolate each pipeline stage within dedicated timeouts (4-10s); stage timeouts or network failures must apply fallback imputations without crashing.	Non-Functional	High
R8	Enforce stateless JWT authorization isolating student, verifier, and admin roles, maintaining immutable audit trails of all verification scans.	Non-Functional	Medium

Evidence: Quiz results on digital forensics and cryptographic hashing; survey papers with gaps identified (Week 2 milestone).

2. System and Detailed Design

2.1 System Design

The system employs a decoupled, asynchronous multi-tier architecture consisting of a **React 18 + Vite + TypeScript** single-page frontend, a **FastAPI (Python 3.11+)** asynchronous REST engine running on Uvicorn, and an **Analytical Forensic & ML Layer** (OpenCV, Tesseract, PyTorch MobileNetV2, Scikit-Learn). Persistent storage uses **SQLite/PostgreSQL** via SQLAlchemy 2.0 ORM for structured scan metadata, disk-backed storage (storage/) for binary uploads and reports, and **Cloud Firestore** for append-only audit logs. This decoupled architecture was chosen because it allows independent scaling of the web client while keeping the compute-heavy computer vision and tensor operations isolated in native C++ optimized Python runtimes.

2.2 Detailed Design Diagrams Included

Diagram	Shows / Architectural Representation
Flowchart	Main logic, decision branches, parallel forensic checks, threshold gating, and verdict assignment.
UML Sequence Diagram	Temporal interactions across Client, API Router, Pipeline Orchestrator, ML Modules, and Relational DB.
DFD (Level 0 Context)	High-level data boundaries between User, System, University Portals, and Admin.
DFD (Level 1 Decomposition)	Detailed flow of data between Processes 1.0–10.0 and Data Stores (Certificates, Templates, Audit Logs).

Detailed Flowchart & Main Logic Representation:

```
Upload [PDF/JPG] -> File Validation (<=10MB) -> SHA-256 Checksum Ingestion
-> OpenCV Deskewing (+-0.5 deg) & CLAHE Enhancement
-> Parallel Analysis:
[1] Tesseract OCR -> Token Extraction -> Field Parsing (Name, USN, Marks)
[2] Error Level Analysis (ELA 90%) & SIFT/ORB Copy-Move Keypoint Matching
[3] QR Code Decode -> Domain Whitelist -> Fuzzy Text Cross-Match
[4] Template Alignment: SSIM (>=0.85) & Canny Edge XOR (<=0.08)
-> 22-Dimensional Feature Vector Extracted & StandardScaled
-> Inference: HistGradientBoosting Trees + MobileNetV2 Deep CNN
-> Multimodal Score Fusion -> Authenticity Score (0-100%)
-> Verdict Decision: [Score >= 80%: AUTHENTIC | 50-79%: SUSPICIOUS | < 50%: FRAUDULENT]
-> Generate ReportLab Audit PDF & Persist to DB -> Deliver JSON to Client
```

2.3 Design Decisions & Alternative Trade-Offs

1. Framework Choice: FastAPI (Python 3.11) vs. Java Spring Boot: FastAPI was chosen because computer vision and deep learning libraries (OpenCV, PyTorch, Scikit-Learn) are implemented in native C++ with zero-copy Python bindings. Using Spring Boot would require continuous, high-latency inter-process communication (IPC) or REST hops between Java and Python, adding 300–800ms overhead. FastAPI provides the same enterprise Controller-Service-Repository modularity within the Python runtime.

2. Machine Learning: HistGradientBoosting / XGBoost vs. Heavyweight Transformers: We deployed Gradient Boosted Trees on 22 tabular features paired with MobileNetV2 for visual patches, rejecting giant Document Transformers (LayoutLMv3). Transformers require GPU VRAM (>2GB) and have latencies >1.5s; boosted trees execute in <15ms on standard CPUs, handle missing values naturally, and provide full mathematical explainability.

3. Database Design: Relational Schema (SQLAlchemy 2.0) vs. Pure MongoDB NoSQL: We chose a relational schema with strict foreign keys because academic certificates, user accounts, and verification jobs have relational integrity requirements (e.g., deleting a certificate must cascade to its OCR fields and forensic findings). SQLAlchemy allows seamless switching from local SQLite to production PostgreSQL.

2.4 Traceability Matrix

Req ID	Requirement Summary	Design Element / Implementation Artifact Satisfying It
R1	Ingestion & SHA-256 Hashing	Process 1.0, backend/app/modules/ingestion/ & verification_hub.py
R2	Preprocessing & OCR Field Parse	Process 2.0 & 3.0, app/modules/preprocessing/ & ocr/service.py
R3	ELA & SIFT Copy-Move Forensics	Process 4.0, app/modules/forensics/ela.py & copy_move.py
R4	QR Whitelist & Cross-Matching	Process 5.0 & 6.0, src/deterministic_checks.py & qr_url_verifier.py
R5	22D Tabular ML & Score Fusion	Process 7.0, 8.0 & 9.0, app/modules/ml_classifier/ & score_fusion/
R6	Sub-1500ms CPU Latency	Lightweight MobileNetV2 depthwise convolutions & in-memory template caching
R7	Graceful Pipeline Fault Tolerance	app/services/fallback_handler.py with run_with_timeout() wrapper
R8	JWT RBAC & Audit Trails	app/core/security.py, deps.py & audit_service.py

Evidence: System Design Document submitted Week 4; Detailed Design Document with diagrams submitted Week 6.

3. Implementation and Functionality

3.1 Progress Against Checkpoints

Checkpoint	Target	What is Working (Features Implemented)	Honest Completion
Core Implementation (Week 8)	~30%	File ingestion, SHA-256 checksumming, PDF rasterization (300 DPI), OpenCV deskewing, basic Tesseract OCR token extraction, initial SQLite schema.	**35%**
Complete, Refine & Debug (Week 10)	~55%	Error Level Analysis (ELA), SIFT/ORB copy-move clustering, QR decoding with whitelist check, 22D feature extraction, baseline ML model training.	**60%**
Consolidation, Phase-1 (Week 14)	~70%	Multimodal score fusion engine, MobileNetV2 visual tamper detector, React 18 verification hub, real-time progress indicators, audit report PDF generator, Vitest/Pytest suites.	**72%**

3.2 Key Features Implemented

1. **Automated Preprocessing & Deskewing:** Detects tilt via contour minimum bounding box analysis and applies affine correction within ±0.5°; applies CLAHE contrast enhancement (clipLimit=2.0). *Source:* app/modules/preprocessing/service.py.
2. **OCR Token Extraction & Structured Parsing:** Extracts tokens and bounding boxes via Tesseract; uses regex to parse student name, USN (\b[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}\b), degree, and date. *Source:* app/modules/ocr/service.py & src/extractors/certificate_extractor.py.
3. **Dual Image Forensics (ELA & SIFT Copy-Move):** Computes pixel error matrices at 90% JPEG re-compression; runs SIFT descriptor clustering to detect cloned stamps and signatures. *Source:* app/modules/forensics/ela.py & copy_move.py.
4. **Deterministic Template Alignment (SSIM & Canny XOR):** Standardizes resolution to (512x512) and checks Structural Similarity (SSIM ≥ 0.85) and Canny edge difference (≤ 0.08) against reference templates. *Source:* src/deterministic_checks.py.
5. **QR Code Accreditation & Cross-Matching:** Decodes QR codes via PyZBar; verifies domain against whitelist; cross-references payload with OCR text using RapidFuzz Levenshtein token matching. *Source:* src/verifiers/qr_url_verifier.py.
6. **22D Tabular Feature Extraction & ML Classification:** Compiles 22 normalized metrics across 5 analytical groups; classifies risk via HistGradientBoosting and MobileNetV2 CNN. *Source:* app/modules/ml_classifier/feature_extractor.py & inference_engine.py.
7. **Multimodal Score Fusion & Audit Reporting:** Synthesizes sub-scores into an Authenticity Score (0-100%); generates official ReportLab audit PDFs with visual bounding-box overlays. *Source:* app/modules/score_fusion/service.py & report_generator/service.py.

3.3 Code Quality and Optimization

The codebase enforces strict static typing with Pydantic V2 on the backend and TypeScript 5.2 on the frontend, passing automated linting via ruff check and eslint. Stage execution is isolated within run_with_timeout() blocks to guarantee resilience.

Significant Optimization Implemented (Reference Template In-Memory Caching):

• *Before:* For every incoming certificate, deterministic_checks.py re-read reference template images from physical disk, resized them, and calculated Canny edge maps from scratch, taking **180–240ms of redundant disk I/O** per verification.

• *After:* Preloaded and cached Gaussian-blurred grayscale matrices and Canny edge arrays for all reference templates into memory during pipeline startup (self.preloaded_templates in unified_inference.py), reducing alignment latency to **< 22ms (an 88% speedup)**.

3.4 Deviations from the Design

• **Replaced Pure OpenCV QR Decoder with PyZBar Primary:** OpenCV's native detector failed on skewed or low-contrast barcodes (<65% recall); PyZBar increased QR detection rate to >98%.

• **Added MobileNetV2 Deep CNN Alongside Tabular ML:** Added a lightweight transfer learning CNN to scan raw pixels directly, catching inpainting artifacts that hand-crafted tabular features missed.

• **Introduced 'SUSPICIOUS / AUDIT REQUIRED' Buffer Tier:** Replaced binary classification (<50% Fake, ≥50% Real) with a 3-tier boundary (50–79% manual audit) to prevent degraded genuine scans from being rejected.

Evidence: Screenshots of core output (Week 8), refined output (Week 10), and integrated modules (Week 14).

4. Testing, Debugging and Integration

4.1 Testcase Design Matrix

Test ID	Feature / Requirement	Input Data	Expected Output	Actual Output	Pass/Fail
T1 (Normal)	Authentic VTU Degree Verification (R1, R2, R4, R5)	High-res genuine certificate (VTU_Degree_01.png), valid QR, matching USN.	Score ≥ 90%; Verdict: AUTHENTIC; Forensic cards green; Audit report generated.	Score: 96.5%; Verdict: AUTHENTIC; Latency: 1120ms; Report compiled.	**PASS**
T2 (Boundary)	Degraded Scan with Crease & Noise (R2, R3, R7)	Genuine certificate scanned at low DPI (100 DPI) with paper crease across text.	OCR confidence drops (0.62); Template SSIM passes; Verdict: SUSPICIOUS (Manual Review).	Score: 71.2%; Verdict: SUSPICIOUS / AUDIT REQUIRED; Warning logged.	**PASS**
T3 (Negative)	QR-Code Hijacking Attack (R4, R5)	Forged certificate with Name B, but pasting Name A's authentic VTU QR code.	QR decodes; Fuzzy cross-matcher flags Name/USN mismatch; Force Verdict: FRAUDULENT.	Critical Flag: QR_DATA_MISMATCH (Match: 28%); Verdict: FRAUDULENT.	**PASS**

4.2 Test Summary

A total of **52 automated tests** were designed and executed across the full stack. The frontend test suite contains **45 tests across 10 test files (100% pass rate)** covering file dropzone bounds, verdict gauges, 3D canvas fallbacks, and admin filtering. The backend test suite contains **7 end-to-end integration tests (100% pass rate)** covering PDF rasterization, OCR extraction, ELA forensics, and API contracts. The test suite achieved >80% line coverage across analytical modules.

4.3 Bugs Found and Fixed

Bug Identified	Root Cause (One Line)	Technical Fix Applied	Evidence
Vite Dependency Optimization Crash (TypeError: Cannot read properties of undefined)	Running npm run dev across an NTFS junction directory caused process.cwd() to mismatch esbuild's realpath lookup.	Resolved canonical path via fs.realpathSync(), ran process.chdir(projectRoot), and removed preserveSymlinks: true.	Vite dev server starts cleanly in 414ms with exit code 0.
Windows Console Encoding Crash in ML Scripts	Scripts printing UTF-8 checkmarks and status symbols crashed on default Windows cp1252 console encoding.	Reconfigured sys.stdout and sys.stderr with UTF-8 encoding and errors='replace'.	Training scripts execute cleanly without console encoding errors.
PDF 72-DPI Degradation Causing OCR Failure	PDF rasterization at default 72 DPI rendered small certificate fonts illegible, dropping OCR accuracy below 40%.	Enforced strict 300-DPI rasterization with INTER_LANCZOS4 in PDFRasterizer.rasterize_pdf().	OCR mean confidence increased from 0.38 to 0.94 on identical PDF inputs.

4.4 Integration of Module-1 and Module-2

Module-1 (Analytical Forensics & Computer Vision Engine) and Module-2 (API Gateway, Business Logic & Web Interface) connect via VerificationPipelineOrchestrator.run_pipeline(), passing in-memory NumPy image buffers through an asynchronous execution context. The main integration issue encountered was CPU thread-blocking during long forensic runs; this was resolved by wrapping CPU-bound OpenCV and PyTorch operations in isolated timeout blocks (run_with_timeout()), allowing them to execute asynchronously without stalling FastAPI's ASGI event loop or disconnecting active HTTP client sessions.

Evidence: Test case execution documents and testing logs (Week 12); bug-fix diffs and root-cause notes (Weeks 10 and 14).

5. Documentation, Phase-2 Plan and Viva Readiness

5.1 My Individual Contribution

Between Weeks 3 and 14, I personally designed, implemented, and tested the end-to-end **Analytical Forensic Pipeline, Machine Learning Engine, and FastAPI Backend Architecture**. Specifically, I authored the 22-dimensional feature extraction engine (feature_extractor.py), implemented Error Level Analysis (ela.py) and SIFT copy-move detection (copy_move.py), trained the MobileNetV2 visual tamper detector and gradient-boosted decision tree baselines, and debugged the Windows NTFS junction Vite server issue. My project partner focused on frontend React component design (Hero3DCanvas.tsx, PublicVerifyPage.tsx), Tailwind styling, and Firebase Authentication setup.

5.2 Phase-2 Pending-Work Plan (Remaining ~30%)

Pending Work Package (About 30%)	Priority	Planned Technical Approach	Target Week
Decentralized Blockchain Ledger Anchor	High	Deploy Ethereum / Polygon ERC-1155 smart contract to anchor SHA-256 certificate hashes on-chain post-verification.	Week 18
Multimodal Document Transformer Integration	Medium	Benchmark fine-tuned LayoutLMv3 models on server GPU nodes for complex unstructured tabular marksheets.	Week 20
Automated University Portal Webhooks	Medium	Implement authenticated webhooks enabling universities to push new degree batches and receive instant revocation notices.	Week 22
Mobile Verifier App (Android / iOS)	Low	Package scanner into a mobile React Native / Flutter client with on-device camera framing and App Check attestation.	Week 24

5.3 Challenges and Learnings

The hardest technical challenge was **distinguishing between legitimate document compression noise and deliberate digital tampering**. Initial forensic runs produced false alarms on authentic WhatsApp-forwarded certificates because lossy JPEG re-saving simulates splicing artifacts. I tried raw Laplacian variance, which proved too noisy; what ultimately worked was combining **CLAHE contrast normalization with dual-signal corroboration**—requiring both localized ELA error jumps and typographical baseline drift before flagging a forgery. If I were to do this differently, I would generate a larger dataset of authentic paper scan variations earlier in the project lifecycle to benchmark noise tolerances prior to feature selection.

5.4 Viva Preparation: Ready to Explain Without Notes

• **Why this design and these technologies, and what were the alternatives?**
FastAPI was chosen over Spring Boot for native C++ memory-mapped execution with OpenCV and PyTorch without multi-process IPC latency. Gradient Boosted Trees were chosen over Document Transformers because they execute in <15ms on commodity CPUs, handle mixed tabular feature scales naturally, and provide full mathematical explainability via Gain and SHAP.

• **How each diagram maps directly to your code:**
The Flowchart maps to pipeline_orchestrator.py executing stages 1 through 10 sequentially; the Sequence Diagram maps to api.ts → verification_hub.py → Orchestrator → ml_classifier → database.py; and the DFD maps to the discrete service folders inside backend/app/modules/.

• **Key algorithm, mathematical logic & computational complexity:**
XGBoost Gradient Boosting Split Gain:
$\text{Gain} = 0.5 * [G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - (G_L + G_R)^2/(H_L + H_R + \lambda)] - \gamma$
where $G_j = \sum g_i$ (gradients / residuals) and $H_j = \sum h_i$ (Hessians / curvatures). Complexity is $O(K \cdot d)$ where K=trees (~150) and d=depth (≤6), executing in <15ms on CPU.

• **Critical bug found and how its root cause was discovered:**
When running npm run dev inside a Windows NTFS junction directory, Vite crashed with TypeError: Cannot read properties of undefined (reading 'imports'). We diagnosed that process.cwd() pointed to the junction while esbuild resolved physical canonical paths. The relative path discrepancy (../../frontend/ vs node_modules/) caused lookup dictionary mismatches. Resolved by removing preserveSymlinks: true and executing process.chdir(fs.realpathSync(...)) in vite.config.ts.

• **What is left to build in Phase-2 and why:**
The remaining ~30% focuses on **Decentralized Blockchain Ledger Anchoring**: deploying an Ethereum/Polygon smart contract to store SHA-256 certificate hashes on an immutable distributed ledger once verified, establishing a globally queryable public proof of authenticity that cannot be revoked or altered by centralized database tampering.

Evidence: Phase-2 Pending-Work Plan submitted Week 14; structured viva voce defense prepared for Week 16 evaluation.